

LECTURE 07

React II, *forms.*

Submit events, controlled inputs, validation, and the browser defaults that surprise new React developers.

forms

controlled inputs

onChange

validation

START WITH HTML

A form groups inputs and fires submit.

```
return (  
  <>  
    <h1>New Post</h1>  
    <form>  
      <input type="text" name="title" />  
      <button type="submit">Create Post</button>  
    </form>  
  </>  
)
```

RENDERED IDEA

THE GOTCHA

Forms want to refresh the page.

By default, a submitted form sends values through the URL or request body and navigates away. React apps usually want to stay on the page.

```
const handleSubmit = (e) => {  
  e.preventDefault() // first line in most React submit handlers  
}
```

prevent the browser default, then run your app logic.

EVENT.TARGET

Named inputs are available on submit.

```
function NewPost() {  
  const handleSubmit = (e) => {  
    e.preventDefault()  
    console.log(e.target.title.value)  
    console.log(e.target.content.value)  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input name="title" />  
      <input name="content" />  
    </form>  
  )  
}
```

GOOD FOR TEACHING

- ✓ Shows the browser event clearly.
- ✓ Requires name attributes.
- ✓ Reads values only at submit time.
- ✓ Less flexible than controlled inputs.

REACT OWNS THE VALUE

value comes from state. onChange updates it.

```
const [title, setTitle] = useState("")

const handleTitleChange = (e) => {
  setTitle(e.target.value)
}

<input
  type="text"
  value={title}
  onChange={handleTitleChange}
/>
```

the input displays state. typing asks state to change.

THE LOOP

Every keystroke takes the same path.

1. USER TYPES

The browser fires a change event.

2. HANDLER RUNS

```
setTitle(e.target.value)
```

3. REACT RENDERS

The input receives the new value.

Controlled inputs feel wordier at first. The payoff is that React can clear, validate, disable, prefill, and submit the form predictably.

SCALING UP

Many fields can share one state object.

```
const [form, setForm] = useState({
  title: "",
  content: ""
})

const handleChange = (e) => {
  setForm({
    ...form,
    [e.target.name]: e.target.value
  })
}
```

TEACHING NOTE

- ✓ Separate state variables are clearer first.
- ✓ Object state reduces repeated handlers.
- ✓ The input name must match the state key.
- ✓ Spread the old object so other fields survive.

BUILT-IN FIRST

Use the browser when it helps.

```
<input
  required
  minLength={8}
  type="password"
  value={password}
  onChange={handlePasswordChange}
/>
```

WHY START HERE?

- ✓ Less JavaScript to write.
- ✓ Built-in accessibility behavior.
- ✓ Great for required fields and simple lengths.
- ✓ Custom validation comes next.

CUSTOM RULES

Errors are state too.

```
const [passwordErrors, setPasswordErrors] = useState([])

const handlePasswordChange = (e) => {
  const nextPassword = e.target.value
  setPassword(nextPassword)

  if (nextPassword.length < 8) {
    setPasswordErrors(["Password must be at least 8 characters"])
  } else {
    setPasswordErrors([])
  }
}
```

validate the next value, not the old state value.

CONDITIONAL UI

Show students why submit is disabled.

```
{passwordErrors.length > 0 && (  
  <div>  
    {passwordErrors.map((error) => (  
      <p key={error}>{error}</p>  
    ))}  
  </div>  
)}  
  
<button disabled={passwordErrors.length > 0}>  
  Create User  
</button>
```

UX RULE

- ✓ Errors should be close to the field.
- ✓ Explain what needs to change.
- ✓ Do not hide the reason for disabled submit.
- ✓ Clear errors when the input becomes valid.

FORMS CHECKLIST

React forms are state plus events.

STEPS

- ✓ Wrap inputs in `<form>`.
- ✓ Use `type="submit"`.
- ✓ Call `e.preventDefault()`.
- ✓ Use `onChange` to update state.
- ✓ Set value from state.

GOTCHAS

- ✗ Missing name when using `event.target`.
- ✗ Forgetting value on controlled inputs.
- ✗ Clearing the form before a request succeeds.
- ✗ Disabling submit without showing errors.